

MyWallet

Application mobile Flutter — Portefeuille financier

COMPTE-RENDU DE PROJET

Mathis Rénac · L3 Informatique

Contexte et ressources de base

Le projet s'appuie sur un dossier initial fourni par l'enseignant, offrant une base technique complète pour le déploiement via Docker.



Docker

docker-compose, Dockerfile Laravel
& Next.js



Backend Laravel

API REST avec fichiers de
configuration existants



Flutter

transfer_screen.dart,
transfer_response.dart,
api_service.dart

Objectif du projet

Développer une application mobile Flutter de type portefeuille financier, inspirée des applications bancaires modernes et de gestion de crypto-actifs.



Gestion des utilisateurs

Inscription et connexion



Transfert d'argent

Envoi de fonds entre utilisateurs



Consultation du solde

Visualisation du solde de l'utilisateur



Fonctionnalités admin

Visualisation des utilisateurs inscrits

AUTHENTIFICATION

Connexion & Inscription

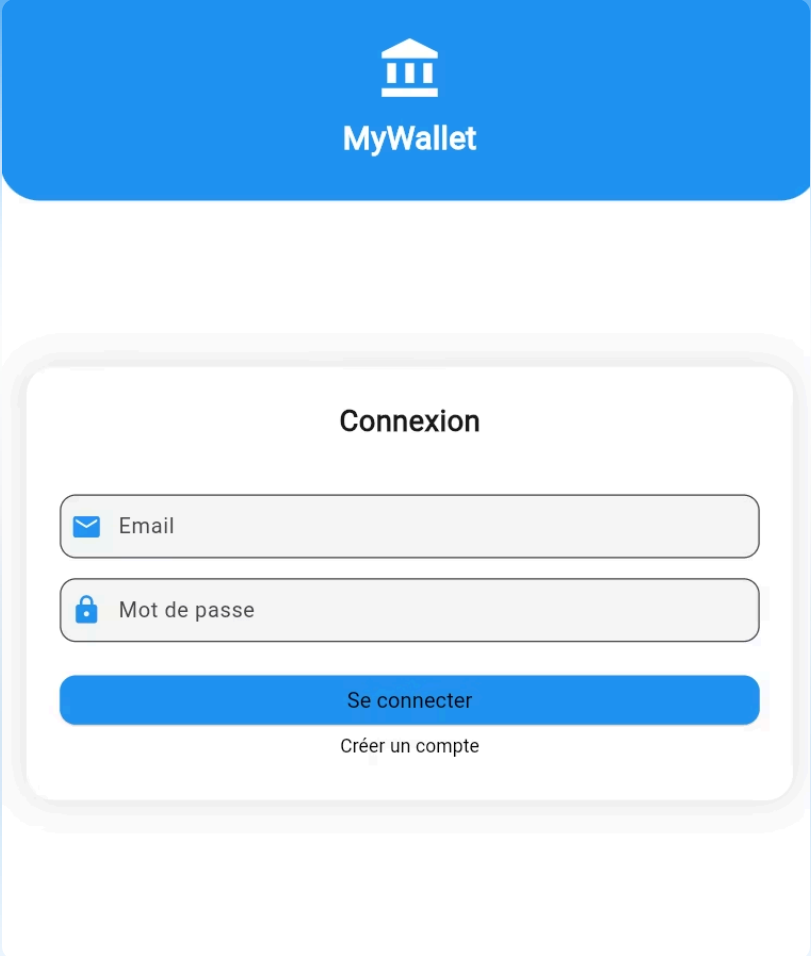
Un écran de connexion complet (`login_screen.dart`) a été ajouté à l'application, permettant la gestion complète des sessions utilisateurs.

Connexion

Authentification via route API
`/login`

Inscription

Création de compte via route
API `/register`



The mockup shows a mobile application interface for 'MyWallet'. At the top is a blue header with a white icon of a wallet and the text 'MyWallet'. Below the header is a white card with a light gray border. The card has a title 'Connexion' in bold. It contains two input fields: the first is labeled 'Email' with an envelope icon, and the second is labeled 'Mot de passe' with a lock icon. Below these fields is a blue button labeled 'Se connecter'. Underneath the button is a link labeled 'Créer un compte'.

INTERFACE ADMINISTRATEUR

admin_screen.dart

Liste des utilisateurs

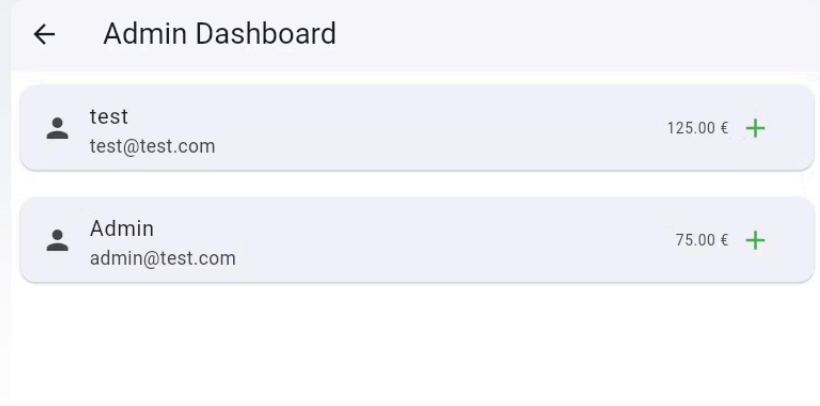
Affichage de tous les comptes enregistrés dans l'application

Visualisation des données

Consultation du nom, email et solde de chaque utilisateur

Opérations de test

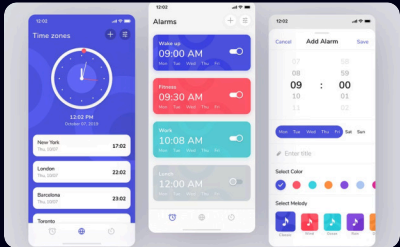
Ajout de fonds sur les comptes utilisateurs pour les tests

A mobile app interface for an Admin Dashboard. It features a back arrow and the title 'Admin Dashboard'. Below is a list of users with their names, emails, and balances. The first user is 'test' with email 'test@test.com' and a balance of 125.00 €. The second user is 'Admin' with email 'admin@test.com' and a balance of 75.00 €. Both balances are followed by a green plus icon.

Admin Dashboard		
	test test@test.com	125.00 € +
	Admin admin@test.com	75.00 € +

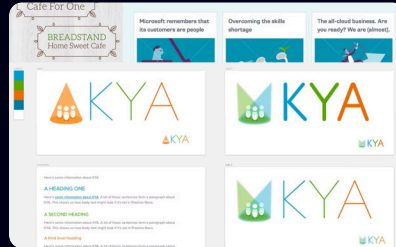
Interface utilisateur & UX

L'ensemble des écrans a été retravaillé pour offrir une expérience utilisateur moderne, lisible et bien structurée.



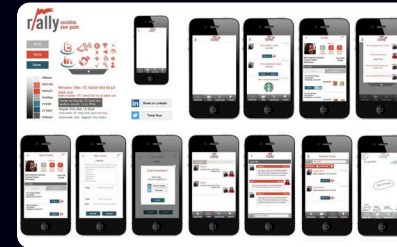
Design modernisé

Refonte visuelle pour un rendu contemporain et professionnel



Lisibilité améliorée

Hiérarchie visuelle renforcée et contrastes optimisés



Organisation structurée

Informations disposées de manière logique et intuitive

FONCTIONNALITÉ CLÉ

Transfert d'argent

L'écran `transfer_screen.dart` a été enrichi pour offrir une expérience complète de gestion des transactions.

→ Historique des transactions

Journal complet des opérations effectuées

→ Envoi d'argent

Transfert de fonds entre utilisateurs

→ Solde dynamique

Mise à jour automatique après chaque opération

→ Déconnexion

Bouton de déconnexion intégré à l'écran

MyWallet



75.00 €

Solde disponible

€ Montant

✉ Email destinataire

Envoyer

Historique

↓ +25.00 €
test@test.com → admin@test.com

↑ -50.00 €
admin@test.com → test@test.com

Adaptation du backend & de l'API

Le backend Laravel et le service Flutter ont été étendus pour assurer une communication complète entre le client et le serveur.

Routes API Laravel (api.php)

/login

/register

/transfer

/users

/transactions

Méthodes Flutter (api_service.dart)

login()

register()

getBalance()

transfererMontant()

getUsers()



Difficultés rencontrées

Plusieurs défis ont été rencontrés lors de l'intégration entre le frontend Flutter et le backend Laravel.

Requêtes HTTP

Problèmes de format et de structure des requêtes envoyées à l'API

Incohérences de données

Divergences entre les données manipulées côté Flutter et celles attendues par le backend

Affichage du solde

Difficultés liées à la récupération et à l'affichage correct du solde utilisateur

Transfert & Admin

Erreurs lors de la mise en place du transfert et de la liste des utilisateurs admin

Conclusion

Ce projet a permis de construire une application complète combinant **Flutter** et **Laravel** via une architecture API REST.

Authentification

Gestion complète des sessions utilisateurs

Communication client/serveur

API REST intégrée et fonctionnelle

Transactions

Transferts et historique opérationnels

Interface moderne

UX soignée et cohérente

 Malgré les défis d'intégration, l'application fonctionne de manière cohérente et remplit les principales fonctionnalités attendues.